

'this' reference in Java

Last Updated : 08 Jan, 2024



In Java, 'this' is a reference variable that refers to the current object, or can be said "this" in Java is a keyword that refers to the current object instance. It can be used to call current class methods and fields, to pass an instance of the current class as a parameter, and to differentiate between the local and instance variables. Using "this" reference can improve code readability and reduce naming conflicts.

Java this reference Example

In Java, this is a reference variable that refers to the current object on which the method or constructor is being invoked. It can be used to access instance variables and methods of the current object.

Below is the implementation of this reference:

```
// Java Program to implement
// Java this reference

// Driver Class
public class Person {

    // Fields Declared
    String name;
    int age;

    // Constructor
    Person(String name, int age)
    {
        this.name = name;
        this.age = age;
    }

    // Getter for name
    public String get_name() { return name; }

    // Setter for name
    public void change_name(String name)
    {
```

```

        this.name = name;
    }

    // Method to Print the Details of
    // the person
    public void printDetails()
    {
        System.out.println("Name: " + this.name);
        System.out.println("Age: " + this.age);
        System.out.println();
    }

    // main function
    public static void main(String[] args)
    {
        // Objects Declared
        Person first = new Person("ABC", 18);
        Person second = new Person("XYZ", 22);

        first.printDetails();
        second.printDetails();

        first.change_name("PQR");
        System.out.println("Name has been changed to: "
                           + first.get_name());
    }
}

```

Output

Name: ABC

Age: 18

Name: XYZ

Age: 22

Name has been changed to: PQR

Explanation

In the above code, we have defined a Person class with two private fields name and age. We have defined the Person class constructor to initialize these fields using this keyword. We have also defined getter and setter methods for these fields which use this keyword to refer to the current object instance.

In the printDetails() method, we have used this keyword to refer to the current object instance and print its name, age, and object reference.

In the Main class, we have created two Person objects and called the printDetails() method on each object. The output shows the name, age, and object reference of each object instance.

Methods to use 'this' in Java

Following are the ways to use the 'this' keyword in Java mentioned below:

- Using the 'this' keyword to refer to current class instance variables.
- Using this() to invoke the current class constructor
- Using 'this' keyword to return the current class instance
- Using 'this' keyword as the method parameter
- Using 'this' keyword to invoke the current class method
- Using 'this' keyword as an argument in the constructor call

1. Using 'this' keyword to refer to current class instance variables

```
// Java code for using 'this' keyword to
// refer current class instance variables
class Test {
    int a;
    int b;

    // Parameterized constructor
    Test(int a, int b)
    {
        this.a = a;
        this.b = b;
    }

    void display()
    {
        // Displaying value of variables a and b
    }
}
```

```

        System.out.println("a = " + a + "    b = " + b);
    }

    public static void main(String[] args)
    {
        Test object = new Test(10, 20);
        object.display();
    }
}

```

Output

```
a = 10    b = 20
```

2. Using this() to invoke current class constructor

```

// Java code for using this() to
// invoke current class constructor
class Test {
    int a;
    int b;

    // Default constructor
    Test()
    {
        this(10, 20);
        System.out.println(
            "Inside default constructor \n");
    }

    // Parameterized constructor
    Test(int a, int b)
    {
        this.a = a;
        this.b = b;
        System.out.println(
            "Inside parameterized constructor");
    }

    public static void main(String[] args)

```

```
{  
    Test object = new Test();  
}  
}
```

Output

```
Inside parameterized constructor  
Inside default constructor
```

3. Using 'this' keyword to return the current class instance

```
// Java code for using 'this' keyword  
// to return the current class instance  
class Test {  
    int a;  
    int b;  
  
    // Default constructor  
    Test()  
    {  
        a = 10;  
        b = 20;  
    }  
  
    // Method that returns current class instance  
    Test get() { return this; }  
  
    // Displaying value of variables a and b  
    void display()  
    {  
        System.out.println("a = " + a + " b = " + b);  
    }  
  
    public static void main(String[] args)  
    {  
        Test object = new Test();  
        object.get().display();  
    }  
}
```

```
}
```

Output

```
a = 10  b = 20
```

4. Using 'this' keyword as a method parameter

```
// Java code for using 'this'
// keyword as method parameter
class Test {
    int a;
    int b;

    // Default constructor
    Test()
    {
        a = 10;
        b = 20;
    }

    // Method that receives 'this' keyword as parameter
    void display(Test obj)
    {
        System.out.println("a = " + obj.a
                           + "    b = " + obj.b);
    }

    // Method that returns current class instance
    void get() { display(this); }

    // main function
    public static void main(String[] args)
    {
        Test object = new Test();
        object.get();
    }
}
```

Output

```
a = 10  b = 20
```

5. Using 'this' keyword to invoke the current class method

```
// Java code for using this to invoke current
// class method
class Test {

    void display()
    {
        // calling function show()
        this.show();

        System.out.println("Inside display function");
    }

    void show()
    {
        System.out.println("Inside show function");
    }

    public static void main(String args[])
    {
        Test t1 = new Test();
        t1.display();
    }
}
```

Output

```
Inside show function
Inside display function
```

6. Using 'this' keyword as an argument in the constructor call

```
// Java code for using this as an argument in constructor
// call
// Class with object of Class B as its data member
class A {
    B obj;

    // Parameterized constructor with object of B
    // as a parameter
    A(B obj)
    {
        this.obj = obj;

        // calling display method of class B
        obj.display();
    }
}

class B {
    int x = 5;

    // Default Constructor that create an object of A
    // with passing this as an argument in the
    // constructor
    B() { A obj = new A(this); }

    // method to show value of x
    void display()
    {
        System.out.println("Value of x in Class B : " + x);
    }

    public static void main(String[] args)
    {
        B obj = new B();
    }
}
```

Output

```
Value of x in Class B : 5
```

Advantages of using 'this' reference

There are certain advantages of using 'this' reference in Java as mentioned below:

1. It helps to distinguish between instance variables and local variables with the same name.
2. It can be used to pass the current object as an argument to another method.
3. It can be used to return the current object from a method.
4. It can be used to invoke a constructor from another overloaded constructor in the same class.

Disadvantages of using 'this' reference

Although 'this' reference comes with many advantages there are certain disadvantages of also:

1. Overuse of this can make the code harder to read and understand.
2. Using this unnecessarily can add unnecessary overhead to the program.
3. Using this in a static context results in a compile-time error.
4. Overall, this keyword is a useful tool for working with objects in Java, but it should be used judiciously and only when necessary.

This article is contributed by **Mehak Narang** and **Amit Kumar**.

'this' reference in Java

 Comment

More info 

Next Article 

static Keyword in Java

Similar Reads

Java Programs - Java Programming Examples

In this article, we will learn and prepare for Interviews using Java Programming Examples. From basic Java programs like the Fibonacci series, Prime numbers, Factorial numbers, and Palindrome numbers to...

Java Basic Programs ▼

Java Pattern Programs ▼

Java Conversion Programs ▼

Java Classes and Object Programs ^

Classes and Objects in Java

In Java, classes and objects are basic concepts of Object Oriented Programming (OOPs) that are used to represent real-world concepts and entities. The class represents a group of objects having similar...

🕒 11 min read

Abstract Class in Java

In Java, abstract class is declared with the abstract keyword. It may have both abstract and non-abstract methods(methods with bodies). An abstract is a Java modifier applicable for classes and methods in Ja...

🕒 10 min read

Singleton Method Design Pattern in Java

In object-oriented programming, a java singleton class is a class that can have only one object (an instance of the class) at a time. After the first time, if we try to instantiate the Java Singleton classes, th...

🕒 8 min read

Interfaces in Java

An Interface in Java programming language is defined as an abstract type used to specify the behavior of a class. An interface in Java is a blueprint of a behavior. A Java interface contains static constants and...

🕒 11 min read

Encapsulation in Java

Encapsulation in Java is a fundamental concept in object-oriented programming (OOP) that refers to the bundling of data and methods that operate on that data within a single unit, which is called a class in...

🕒 8 min read

Inheritance in Java

Java, Inheritance is an important pillar of OOP(Object-Oriented Programming). It is the mechanism in Java by which one class is allowed to inherit the features(fields and methods) of another class. In Java,...

🕒 13 min read

Abstraction in Java

Abstraction in Java is the process in which we only show essential details/functionality to the user. The non-essential implementation details are not displayed to the user. In this article, we will learn about...

🕒 11 min read

Difference Between Data Hiding and Abstraction in Java

Abstraction Is hiding the internal implementation and just highlight the set of services. It is achieved by using the abstract class and interfaces and further implementing the same. Only necessarily...

🕒 6 min read

Polymorphism in Java

The word 'polymorphism' means 'having many forms'. In simple words, we can define Java Polymorphism as the ability of a message to be displayed in more than one form. In this article, we will...

🕒 6 min read

Method Overloading in Java

In Java, Method Overloading allows different methods to have the same name, but different signatures where the signature can differ by the number of input parameters or type of input parameters, or a...

🕒 9 min read

Overriding in Java

In Java, Overriding is a feature that allows a subclass or child class to provide a specific implementation of a method that is already provided by one of its super-classes or parent classes. When a method in a...

🕒 13 min read

Super Keyword in Java

The super keyword in Java is a reference variable that is used to refer to parent class when we're working with objects. You need to know the basics of Inheritance and Polymorphism to understand the...

🕒 8 min read

'this' reference in Java

In Java, 'this' is a reference variable that refers to the current object, or can be said "this" in Java is a keyword that refers to the current object instance. It can be used to call current class methods and field...

🕒 6 min read

static Keyword in Java

The static keyword in Java is mainly used for memory management. The static keyword in Java is used to share the same variable or method of a given class. The users can apply static keywords with variables...

🕒 9 min read

Access Modifiers in Java

In Java, Access modifiers helps to restrict the scope of a class, constructor, variable, method, or data member. It provides security, accessibility, etc. to the user depending upon the access modifier used with...

🕒 6 min read

Java Methods Programs



Java Searching Programs



Java 1-D Array Programs



Java 2-D Arrays (Matrix) Programs



Java String Programs



Java List Programs



Java Date and Time Programs



Java File Programs



Java Directory Programs



Java Exceptions and Errors Programs



Java Collections Programs



Java Multithreading Programs



Article Tags :

[Java](#)[School Programming](#)[java-basics](#)

Practice Tags :

[Java](#)